John McCarroll
August 5th, 2024
CSCI 799 - Computer Science Graduate Independent Study
PhD. Richard Lange

Satellite Inspection Dynamics Models: Final Report

Introduction:

Throughout the duration of the 2024 Summer semester, this independent study focused on investigating the application of dynamics models to a Satellite Inspection Reinforcement Learning (RL) environment. This research can be separated into two components: a review of literature related to Model Based Reinforcement Learning (MBRL) methods and a practical application of machine learning techniques on data collected from the Satellite Inspection environment. We found that simple dynamics models struggled to approximate the transition function of the environment, due to features specific to the environment's observation space such as varied scales of feature values, nonlinearity of transition dynamics, and partial observability. These errors compounded when models were used recurrently to predict multiple time steps into the future, raising concerns about the efficacy of these models on planning algorithms for this problem (a common advantage enjoyed by MBRL algorithms). In the remainder of this paper, we provide a summary of the literature reviewed in this course, a description of the Satellite Inspection environment in question, an overview of the iterative process of designing dynamics models for this environment, and summarize key takeaways. The code repository for this project is hosted on GitHub and can be found here.

MBRL Literature Review:

RL is a Machine Learning (ML) method in which a model learns a global policy and/or value function by interacting with an environment, formalized as a Markov Decision Process (MDP). In this context, a policy is a function that maps observations of the current state of the environment to actions for an agent to execute. A value function maps the current state of the environment to a scalar value of how beneficial it is to the agent's goals (often an estimate of expected future rewards). MBRL is a subset of RL in which a world model is used to simulate future states of the environment in a process called planning, in efforts to learn global policy or value functions (Sutton, 2018).

A MDP is defined as a tuple of (S, A, P_a, R_s) where S is the state space, A is the action space, $P_a(s_{t+1} \mid s_t, a_t)$ is a transition function which defines the likelihood that taking action a_t in state s_t will lead to state s_{t+1}, $R_a(s_t)$ is a reward function which returns the immediate rewards associated with the given state s_t. A dynamics model in MBRL refers to a function which performs state transitions. In environments with known dynamics, this can be the actual P_a function. Often, in complex and real-world environments, the true dynamics are unknown, in which case a model must learn to approximate the true P_a function. Reward models refer to functions that compute the immediate reward associated with a given state (R_a). In some safety critical environments, such as the Safety-Gymnasium (Huang, 2024) suite, an additional "cost" term is added to the MDP tuple. This term is a function of the current state, $C_s(s_t)$, and returns a scalar measure of how dangerous the given state, s_t, is. The set of dynamics and reward (and sometimes cost) models are referred to as a "world model". The term policy refers to a function that maps the current state, s_t, to an action, a_t, $\pi(s_t) = a_t$. Often, policy functions are implemented as Deep Neural Networks (DNNs) and are optimized to maximize expected discounted reward. Policies can be global, meaning that they can be applied to the entire state space, or local, meaning that they are designed for a small segment of the state space.

MBRL algorithms leverage their world models to engage in planning. The purpose of planning is to improve the policy, either directly or indirectly. Some algorithms employ planning algorithms such as Monte Carlo Tree Search (Schrittwieser, 2020), Cross Entropy Method (Hafner, 2019), or Model Predictive Path Integral (Hansen, 2023) to search for an optimal action for each environment interaction. These are examples of "decision-time planning" (Sutton, 2018). The goal of decision-time planning is to optimize a local policy for the given state. Other MBRL algorithms engage in "background planning", like Dyna-Q or Counterfactually-Guided Policy Search (CFGPS) (Sutton, 2018)(Buesing, 2018). These algorithms use planning to optimize an approximate value (or Q) function, in an attempt to *indirectly* improve their global policy's decisions. In high dimensional and unknown environments, the quality of an algorithm's plans tends to be bottlenecked by the accuracy of its approximate world model. Model errors can be exploited, leading to degradation in the resulting local policy (Lam, 2021)(Ha, 2018). To some extent, stochastic dynamics models have been applied to mitigate these issues (Hafner, 2023) (Ha, 2018). Stochasticity prevents the policy from overfitting an error-prone, optimistic model, as these errors are not consistently reproducible. This requires the learned policy to be more general and, by extension, generalize better to the actual environment. Stochastic world models are also better suited to model stochastic environments. In more recent works, some researchers have experimented with enabling DNNs to interpret the results of planning

(simulated trajectories or trees of potential game states), in an attempt to mitigate model errors and exploits. The idea is to enable a policy function to determine whether the plan is useful and practical before acting on it (Racanière, 2017) (Chung, 2024).

Another consideration in MBRL for high dimensional environments is how to represent the current state. Many algorithms make use of an “encoder” model, often trained like a Variational Autoencoder (VAE), in order to compress high dimensional observations (like images) into a vector embedding space (also called a “latent space”). It is common for modern MBRL algorithms to learn world models, and even policy functions, on this embedding space, rather than the full observation space (Hafner, 2023) (Hansen, 2023) (Ha, 2018) (Deng, 2022) (Hafner, 2019) (Schrittwieser, 2020)**,** (Lam, 2021)). This raises the question of how to design the latent space to optimize representation capacity. MuZero and the algorithm debuted in *World Models* simply use a vector of float values. DreamerV3 uses a Recurrent Stochastic State Machine (RSSM) world model, whose latent space contains two components: a deterministic vector of floats and a stochastic matrix of one-hot encodings. TD-MPC2 uses a square matrix of vectors, where each vector is normalized using the simnorm operation (Hansen, 2023). Another consideration involves the loss function for the encoder module. The algorithm debuted in *World Models* simply uses a decoder module and a reconstruction loss, a common paradigm for VAEs. DreamerV3 uses the RSSM latent state as input to its policy, reward, and value models, adding learning signals for its encoder and dynamics models from downstream reward prediction, value prediction, and reconstruction tasks. TD-MPC2 and MuZero authors argue that reconstruction losses waste model capacity on a task that is not required for planning. They elect to train their encoders to minimize reward and value prediction error, and achieve convincing results. For a high level comparison of the design decisions made in each paper reviewed for this course, see Table 1.

MBRL algorithms contain several highly coupled components, all with a myriad of potential design decisions. Alterations to state representations can affect the efficacy of world models, which affects planning, which affects policy learning, etc.. Figure 1 contains a useful high-level representation of how each of the components of MBRL algorithms influence one another. Compared to recent successes of Model-Free RL (MFRL), designing and training these additional components seems needlessly onerous. However, it is not a fruitless labor. MBRL exhibits many advantages compared to MFRL. The first, and most obvious, is that access to a world model enables planning. Planning can greatly improve the sample efficiency of RL algorithms. MBRL algorithms consistently achieve comparable performance to MFRL algorithms on the same environments with an order of magnitude less environment interactions. This can

be highly beneficial on tasks where collecting environment interactions is expensive, time consuming, or dangerous. Many real world environments have these characteristics, which limits MFRL techniques' applicability to real world problems. A consequence of decision-time planning is that the resulting decision tree or simulated trajectories can be visualized for a human collaborator, increasing interpretability. DNNs are too complicated for humans to intuitively understand, which limits our ability to properly calibrate when to trust them. This is a major limitation for Deep RL (DRL) algorithm's adoption and application to safety-critical problems (like motor vehicle operation). Providing decision-time plans mitigates this issue by enabling humans to vet a model's decision making process before it is acted out. Another benefit of learning a world model is generalizability. Many environments share similar transition functions. All control tasks in the real world are governed by the same laws of physics, for example. Learned dynamics models can often be applied to various tasks, further increasing data efficiency for learning task-specific policies (Walker, 2023). Another argument in favor of MBRL appeals to the analogy of neuroscience. Humans all experience an internal model of their world and various environments they engage with, from a basketball court to the stock market. We use these models to plan into the future and attempt to make optimal decisions in the current moment. CFGPS takes further inspiration from neuroscience to incorporate counterfactual reasoning into the policy learning process. The authors use a Structural Causal Model (SCM) to fit the MDP of a given environment. They use past environment interactions to approximate posterior distributions of the causal mechanisms (probabilistic functions relating various elements of the MDP) in the SCM, then perform counterfactual inference (substitution of past policy causal mechanisms for alternative policies). The result is a reduction in world model bias during background planning, attributable to the use of posterior distributions rather than priors, which greatly improved sample efficiency. CFGPS was capable of learning a successful policy for a partially observable variation of Sokoban with only a single episode of data (50 timesteps) (Buesing, 2018).

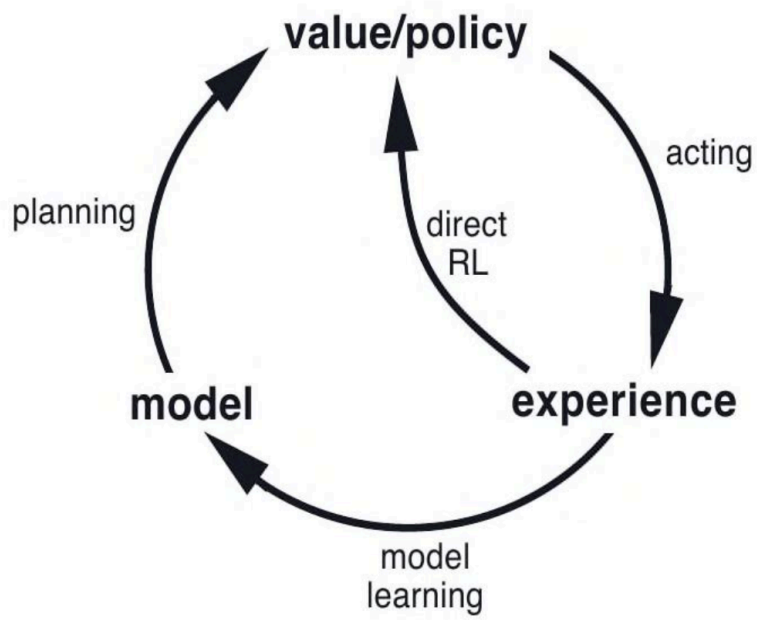


Figure 1. (Sutton, 2018)

Environment:

The satellite inspection environment used throughout this project is a product of research conducted by Air Force Research Labs' Safe Autonomy Team (Ravaioli, 2022). The team has developed lightweight simulations of satellite operations which serve as benchmark environments for researching the application of RL techniques for automated satellite control. For this project, we focused our efforts on the Translational Satellite Inspection environment, in which an agent controls a spacecraft, the "deputy", in a natural motion circumnavigation relative orbit about a second, non-controlled, spacecraft, the "chief". The orbital dynamics assume a low-earth orbit. The agent's goal is to maneuver the "deputy" spacecraft around the "chief" spacecraft such that 100% of the chief's surface could be photographed for inspection. This type of maneuver is useful in the context of collecting information on a damaged spacecraft before attempting repairs. Each timestep in the environment equates to 10 seconds of wallclock time. The environment also models light from the sun, which illuminates one face of the chief's surface. The chief's surface is represented by a sphere of 100 points around the chief. Points on the chief's surface are "inspected" once they are illuminated and within the line of sight of the deputy. The agent controls the deputy spacecraft using three dimensional thrust controls. The action space, then, can be defined as a three element array of float values in the range of [-1, 1]. The agent's observation space contains several features of the environment's state: the deputy's position, the deputy's velocity, the number of inspected points, the position of the center of the cluster of uninspected points, and the angle of the sun.

Data Preprocessing:

To generate the transition data required to train dynamics models for this environment, we first trained a model-free agent using PPO for one million timesteps. We then collected about one million timesteps of the trained PPO agent interacting with the Satellite Inspection environment. We randomly selected and segregated about 100 thousand timesteps to create a test dataset and another 100 thousand timesteps to create a validation dataset. The remaining 800 thousand timesteps constituted our training dataset.

We found it helpful to preprocess the observations data before training dynamics models, given that some features are different scales. Position values range from -800 m to 800 m while the uninspected points cluster position values are normalized to range from -1 to 1, for example. We normalized all observation values to ranges between -1 and 1 (with the exception of the number of inspected points, which is never negative and was normalized between 0 and 1). The sun angle is originally defined in radians, which causes a discontinuity that can be difficult for neural networks to handle (-π and +π represent the same angle). So we converted the sun angle into the x and y positions of a point on the unit circle which corresponds to the provided sun angle. This results in a new observation space definition for training, which contains 12 elements ranging between -1 and 1.

Dynamics Models:

Dynamics models are trained to predict future observations, provided the current observation and action. In this project, all models are trained to predict a single timestep into the future. Accordingly, all models take a 15 element vector input and output a 12 element vector, where the input is a concatenation of the 12 element observation and 3 element action. Formally, $D(obs_t, a_t) = obs_{t+1}$, where D is the dynamics model, obs_t is the observation at timestep t, a_t is the trained agent's action at timestep t, and obs_{t+1} is the predicted observations at timestep t+1. We elected to use the mean-squared error loss function, comparing the model's output to the actual observations at t+1: $loss_{MSE} = \frac{1}{n}\sum_{i=1}^{n}(obs_{t+1}[i] - obs_{t+1}[i])^2$, where obs_{t+1} is the actual observation at timestep t+1 from the dataset. The first dynamics model trained was a simple multilayer perceptron (MLP) with three hidden layers of size 256 and ReLU activation functions applied to the output of each layer. To evaluate the model, trajectories are selected from the test dataset and the model is tasked with predicting multiple timesteps into the future. This is accomplished by first feeding the ground truth observations from the trajectory into the

model and storing the output predictions for the next timestep. These predictions are then concatenated with the next actions applied in the actual trajectory, creating the next observation-action vector. These vectors are subsequently input to the model. This process repeats until the end of the trajectory. Errors (measured in Euclidean distance) are then computed and stored. Multistep predictions of this kind are often used in planning and policy learning in MBRL algorithms, motivating the selection of this evaluation protocol for the project.

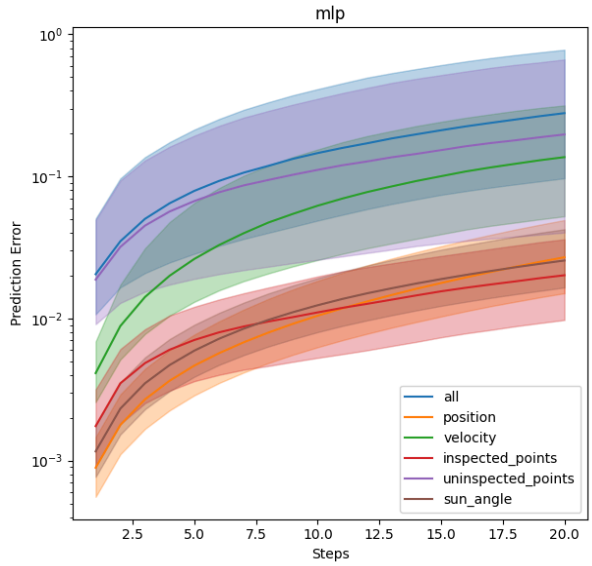


Figure 2.

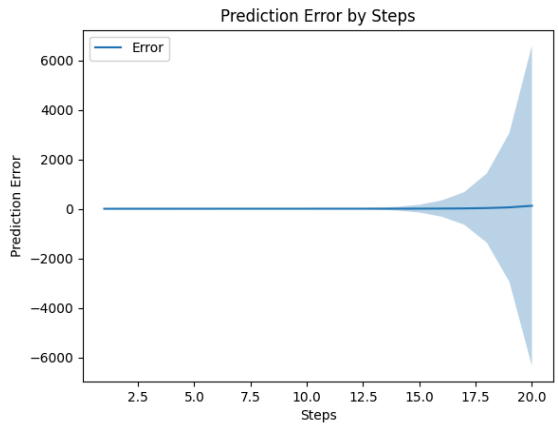


Figure 3.

Results of these initial trials are displayed in Figures 2 and 3. The average error of single step predictions for simple MLPs was about 1.02. The largest contributor to the error was the segment of the observation space representing the xyz position of the uninspected points cluster, followed by the deputy's velocity. We observed that the average prediction error compounds aggressively throughout a multistep trajectory (Figure 3), leading to unreliable and unrealistic predictions as little as five steps in the future. This severely limits this model's applicability to MBRL algorithms.

To improve upon these benchmark results, we first investigate the hypothesis that a simple MLP may not have the capacity to model the transition dynamics of this environment with high fidelity. A simple test to check if model capacity is the limiting factor is to increase the size of the model. We built a larger model, a similar 3 layer MLP where the layers are of size 1024 and the ReLU activation function is applied to the output of each layer. As seen in Figure 4, the larger MLP actually performed slightly worse at multistep predictions. This suggests that model

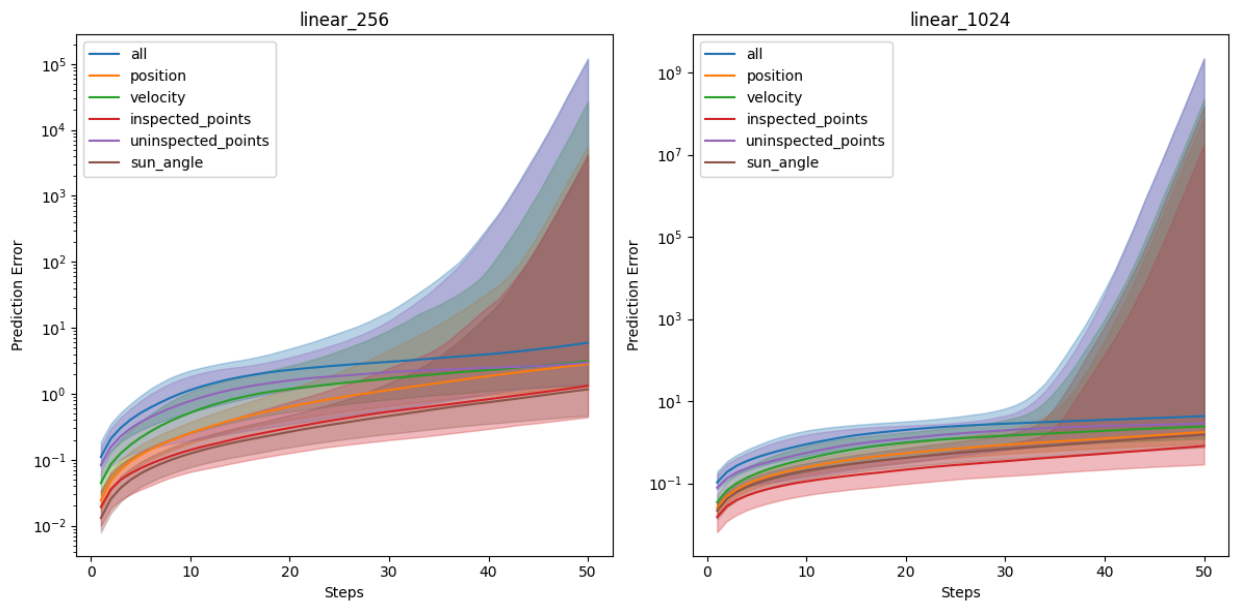


Figure 4.

capacity is not the limiting factor and that the larger model may have overfit the training data to a greater extent.

The satellite inspection environment contains nonlinear dynamics to model the orbital dynamics of the deputy around the chief and the application of thrust to the deputy spacecraft. Motivated by the compounding error and relatively high velocity prediction error we next hypothesized that the MLP is not able to accurately model the nonlinear dynamics of the environment. In an attempt to address this issue, we altered the definition of our model. Rather than predicting the absolute next observation, the next generation of dynamics models predict the *change* in observation values between timesteps. Formally, $D(obs_t, a_t) = \Delta_{obs}$, where Δ_{obs} is the difference between obs_{t+1} and obs_t. To use these models in practice, the model's output is added to the input observations before calculating loss or error: $obs_{t+1} = D(obs_t, a_t) + obs_t$. The results of the Delta MLP model are displayed in Figure 5. This model performs slightly better than the baseline MLP, when predicting 8 or less timesteps into the future. The average single step error for the Delta MLP was about 1.01. Regions of the observation space contributed differently to the total error. Notably, the uninspected points cluster location and number of inspected points prediction errors were reduced, while the deputy's position prediction error was increased. Velocity prediction error remained relatively unchanged. Interestingly, the Delta MLP's performance in longer horizon multistep predictions decreased (where number of steps greater than 8) compared to baseline. While encouraging, the improvements from the Delta

MLP model are marginal and such a model would still limit the success of MBRL algorithms in this environment (especially with degraded long horizon multistep predictions).

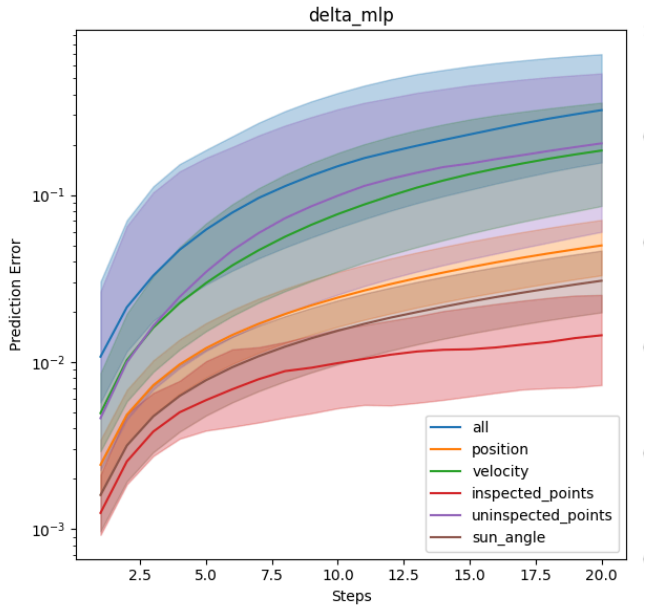


Figure 5.

The compounding error observed in long horizon multistep predictions of our models thus far can be attributed to instability in the model's predictions. After visualizing some Delta MLP predicted trajectories alongside actual trajectories (Figure 6) it became clear that error accumulates in a series of multistep predictions, until, at some point, the model's predicted next observation is far enough out of distribution from the training data as to cause the next predicted observations to catastrophically error. These catastrophic prediction errors involve observation values that are impossible in the given environment. In Figure 6, one can observe the model predicting the number of inspected points decreasing, the deputy's position exceeding the boundaries of the environment, and the uninspected points cluster location moving far away from the chief, none of which are possible in the environment. Note, the y axes in Figure 6 are scaled by each observation element's z score with respect to the entire dataset. In order to address these catastrophic predictions and therefore reduce the error of long horizon multistep predictions, we next implemented a constraint function which applied domain knowledge to the model's output to ensure all predictions were, at a minimum, possible given the environment's transition dynamics.

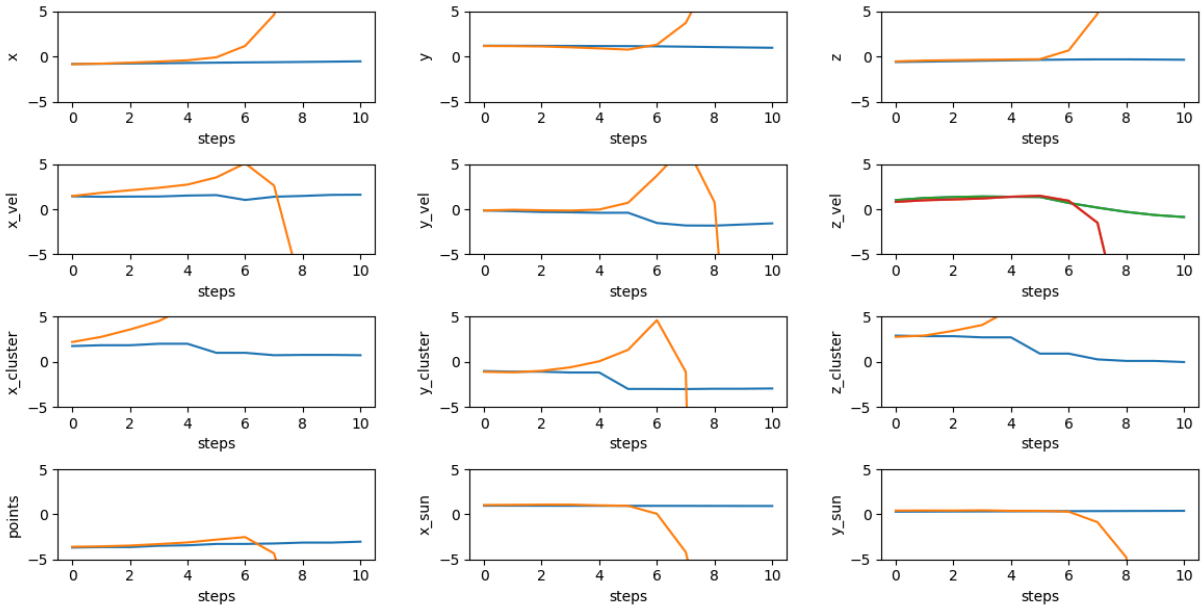


Figure 6.

The first constraint is to limit each segment of the model's output observation space to the ranges defined in our dataset. We use the hyperbolic tangent function to transform the values of the model's outputs to the range of (-1, 1). The exception here is the element of the observation space which corresponds to the number of inspected points, which cannot be negative. In this case, we use the sigmoid function to constrain the output to the range of (0, 1).

The second constraint is to limit the prediction of the deputy's position to values to reside within the boundaries of the environment. The satellite inspection environment allows the deputy to travel a maximum distance of 800 meters from the chief (which is situated at the origin). Since the values in our dataset have been normalized, we check if the model's position prediction vector exceeds a magnitude of 1. If it does, we normalize the vector so that the predicted position point lies on the unit sphere:

$$pos_{constrained} = pos \div |pos|, \qquad \text{if } |pos| > 1$$

The third constraint is to ensure the predicted number of inspected points has not decreased. For this, the model's output number of inspected points is compared to the model's input number of inspected points. If the output is lower than the input, the difference is added back to the output.

$$points_{constrained} = points + (points_{input} - points), \text{ if } points_{input} - points > 0$$

The fourth constraint is to ensure the uninspected points cluster location never moves towards the agent. In an environment where the deputy can observe and inspect roughly one half of a spherical cluster of points, the center of the uninspected points cluster will lie with the

half of the sphere opposite the deputy. As the deputy maneuvers around the sphere, inspecting more points, the uninspected points cluster will move, but never towards the deputy, as this would require the addition of inspected points in the half of the sphere which is observable to the deputy. In order to enforce this constraint, we compute two vectors: one which captures the change in cluster position (delta vector) and another that captures the distance between the previous cluster position and the previous deputy position (deputy vector). The deputy vector is normalized. The delta vector is projected onto the deputy vector. If this projection is positive, then the predicted cluster position has moved closer to the agent. In this case, the component of the change in cluster positions that corresponds to the deputy vector is subtracted from the predicted cluster position.

$$cluster_{constrained} = cluster_{predicted} - proj_{delta}(deputy) * \frac{delta}{|delta|},$$

if $deputy \bullet delta > 0$

Where $deputy = position_{input} - cluster_{input}$

$$delta = cluster_{output} - cluster_{input}$$

$$proj_{delta}(deputy) = ((deputy \bullet delta) / |deputy|)\, deputy$$

The fifth constraint is to ensure the predicted uninspected points cluster position lies within the unit sphere. This is applied after the prior constraint. We use a similar formula to the position constraint.

$$cluster_{constrained} = cluster \div |cluster|, \qquad \text{if } |cluster| > 1$$

The sixth and final constraint projects the sun angle prediction onto the unit circle via normalization, similar to the position constraint.

$$angle_{constrained} = angle \div |angle|$$

We applied the constraints to all previous models during training and evaluation. The results are displayed in Figures 6 and 7. Unfortunately, the constraints seemed to degrade overall performance for both the baseline MLP and the Delta MLP models. The uninspected points cluster and sun angle prediction errors both increased, compared to unconstrained performance, with uninspected points remaining the highest contributor to prediction error. The inspected points prediction error actually improved quite a bit for predictions a few timesteps into the future, but eventually exceeds the prediction errors of unconstrained model's in longer horizon multistep predictions. The degradation of model performance after applying the constraints suggests that some of our constraint assumptions may be incorrect or that the

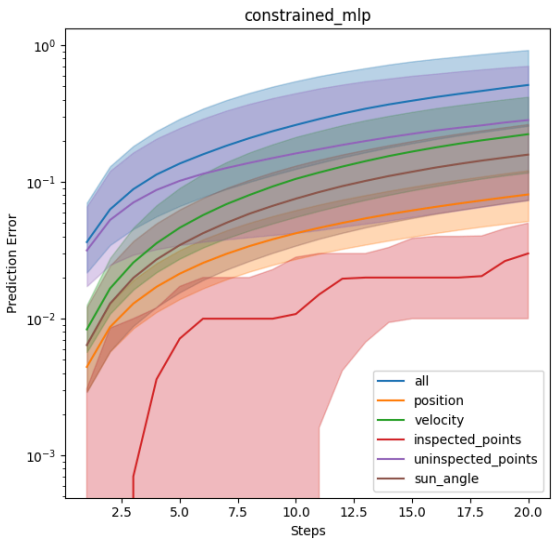


Figure 7.

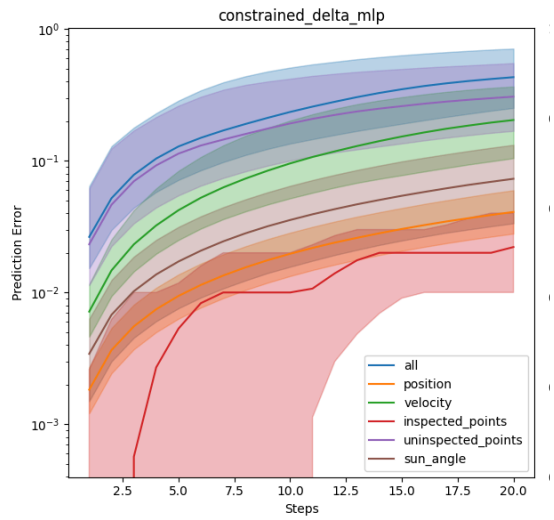


Figure 8.

models may be overconstrained. An ablative investigation into the constraint function was outside of the scope of this project, but could likely improve these results.

Another hypothesized source of model error focuses on the partial observability of the Satellite Inspection environment. In all the model results thus far, the highest contribution towards prediction error has been the uninspected points cluster position. In order to compute this feature of the environment you need access to information not provided in the observation space, namely a full list of every inspection points' position and status (either "inspected" or "uninspected"). One common technique for handling partial observability is to use a Recurrent Neural Network (RNN) model. These models take in an additional "hidden state" embedding vector along with the original observation-action vector as input. They also output a hidden state embedding vector, which is passed to the model as input during the model's prediction on the next input in the time series dataset. These networks excel on time series data (like RL trajectories) and address partial observability by allowing the model to encode and reused important information from past inputs via the hidden state vector. In order to train RNNs, the previous training paradigm needs an update. To handle the initialization of the hidden state, models must be fed each trajectory in sequence. In the new version of the training script, trajectories are batched together and the model predicts the next steps in sequence, starting at t = 0. The MSE loss is computed and a single backpropagation step is performed once per batch of trajectories. This results in significantly larger training batches for the RNN models, although initial trials with smaller batch sizes failed to improve model performance.

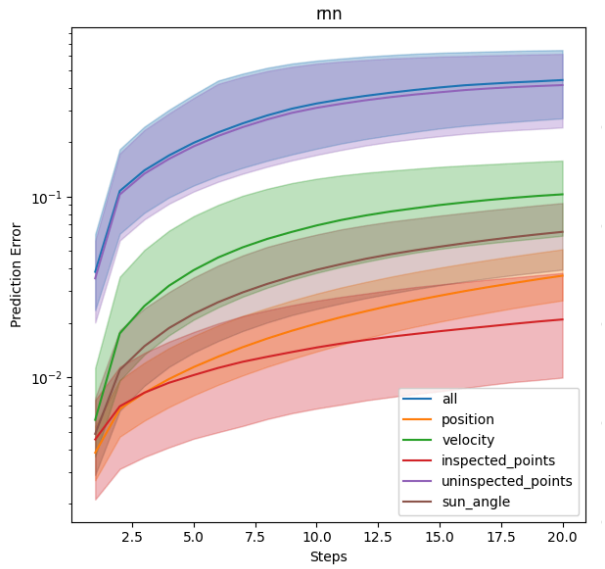


Figure 9.

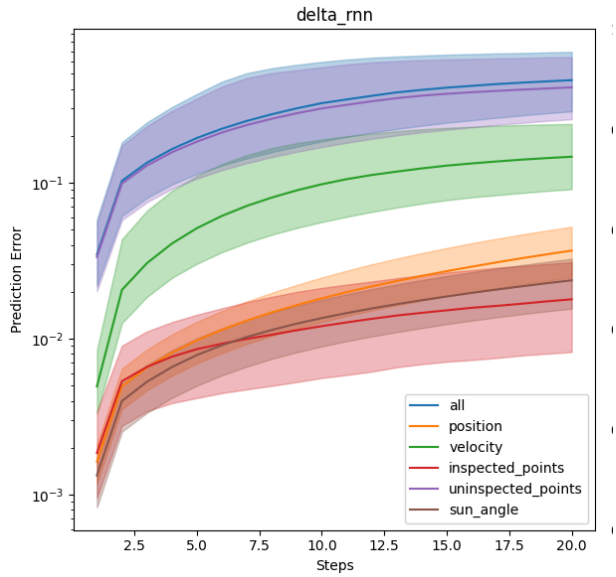


Figure 10.

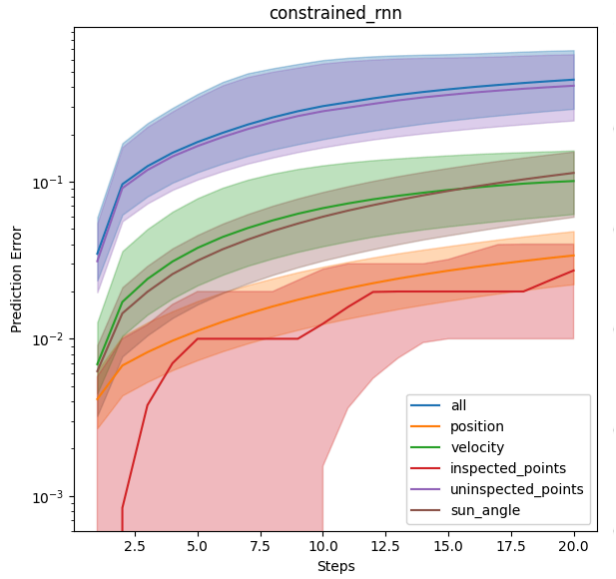


Figure 11.

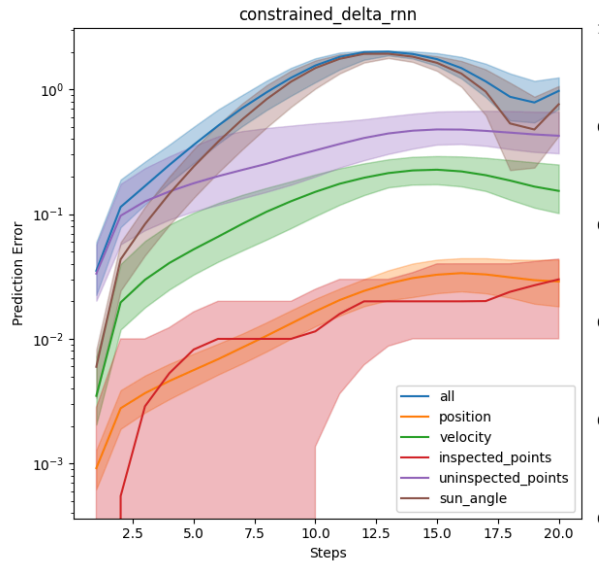


Figure 12.

Unfortunately, all of the RNN models trained on this dataset failed to perform better than the baseline MLP. The Delta RNN model was the best performing RNN model, albeit slightly, with an average one-step prediction error of about 1.01. Consistent with previous results, the uninspected points cluster position remains responsible for the largest contribution to prediction error, demonstrating that the RNN models tried were not better suited to handle the partial observability of that fickle environment feature. Again, we see that the application of constraints to these models further degrades performance. For both the Constrained RNN and the Constrained Delta RNN, sun angle prediction error increased. In the latter case, there is an

emergent, cyclic pattern of increasing then decreasing sun angle prediction error. In this dataset, the sun angle is represented by a point which steadily rotates about the unit circle. The cyclic errors demonstrate the model's learned overreliance on the constraint function, which projects the model's prediction onto the unit circle. This disrupts the learning signal, preventing the model from learning to predict points near the unit circle. The cyclic nature of the prediction error suggests that the model's predictions are projected onto a similar point on the unit circle each timestep, and the error increases or decreases as a function of the sun's orbit.

Failure of the prior two iterations of dynamics models to improve upon benchmark results suggests that more sophisticated model architectures are required to successfully approximate the transition function of this environment. We decided to build probabilistic models for this data. Rather than output a single prediction, probabilistic models learn to output a distribution over the prediction space. Given that some segments of the observation space have high error rates, a probabilistic model will be able to adjust the confidence of its predictions accordingly, assigning these segments a high variance (low certainty). While probabilistic models tend to excel in stochastic environments (and the Satellite Inspection environment is deterministic), the ability to adjust prediction confidence could keep the model's loss function low in the face of uncertainty, reducing the accumulation of compounding errors during multistep predictions.

To create probabilistic models, we extended our four prior models: MLP, Delta MLP, RNN, and Delta RNN. For each model, we doubled the size of the output vector, so that each model outputs a vector of 24 elements. The first 12 elements of each output vector are interpreted as the means of a Gaussian distribution, while the last 12 elements are interpreted as the log variances. Here we define log variance as: $logvar = log(\sigma^2)$. During training, the MSE loss function was substituted for negative log likelihood:

$$loss_{logprob} = -\sum_{i=0}^{d} e^{-\frac{1}{2}(x_i-\mu_i)^T \Sigma_i^{-1}(x_i-\mu_i)-\frac{1}{2}log|\Sigma_i|-\frac{d}{2}log2\pi}$$

Where x = obs_{t+1},

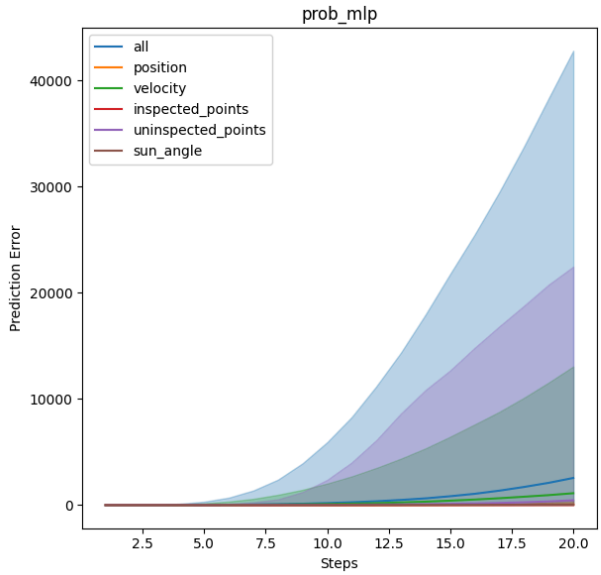
μ = the means predicted for timestep t+1,

Σ = the diagonal covariance matrix constructed from the model's log variances for timestep t+1,

and d = the dimensionality of the observation space = 12.

While evaluating probabilistic models for multistep predictions, the next observations (used for the observation-action input to the model in the next time step) were selected via sampling from the model's output Gaussian distributions. In this way, these dynamics models

are used to stochastically predict the future, similar to stochastic models popular in literature. Prediction error for these models is measured using negative log likelihood during evaluation. In order to compare these models to baseline non-probabilistic models, log likelihood must be calculated in a slightly different way. The average MSE for a non probabilistic model is first computed over the test dataset, then used to populate the log variances for the model's prediction, before calculating negative log likelihood.



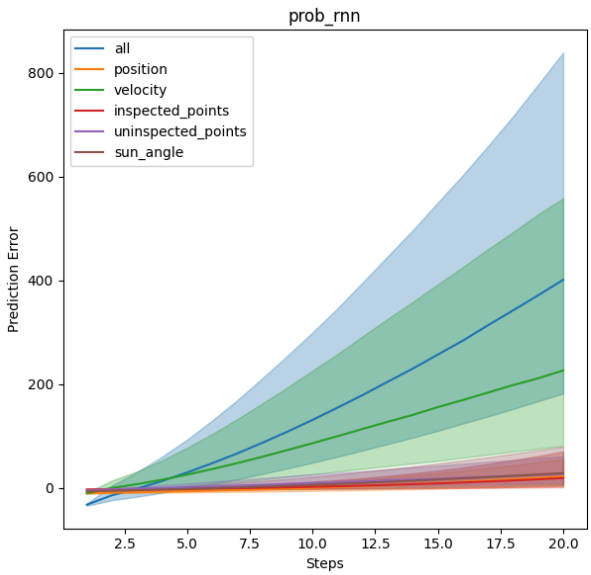


Figure 13. Figure 14.

Figures 13-20 display the results of probabilistic models and their non-probabilistic counterparts. The Probabilistic Delta RNN and Probabilistic RNN performed the best of the probabilistic models, although that's not to say they improved upon benchmark model results. Non-probabilistic models have negative log likelihood scores orders of magnitude lower than their probabilistic counterparts. The most extreme case is the comparison of Delta MLP (Figure 19) and Probabilistic Delta MLP (Figure 15). While Delta MLP is able to maintain negative scores (positive log likelihood), the Probabilistic Delta MLP results quickly explode in the positive direction. In all probabilistic models, we see an exacerbation of the compounding of prediction errors. This could suggest that stochastic sampling from the model's output is ill-fit for the deterministic data collected from the environment.

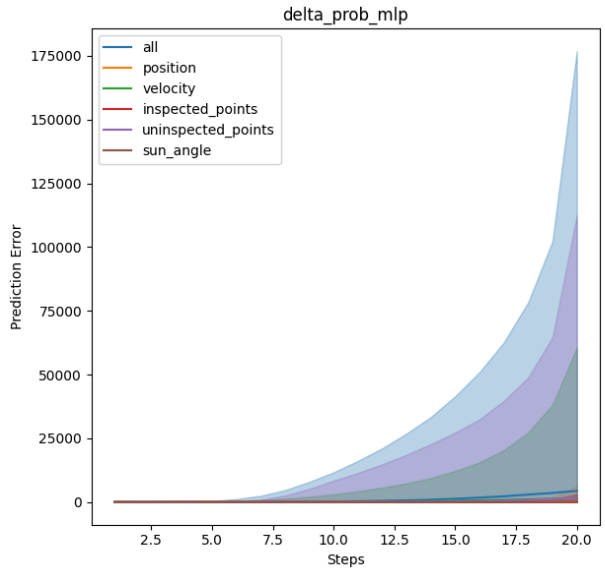


Figure 15.

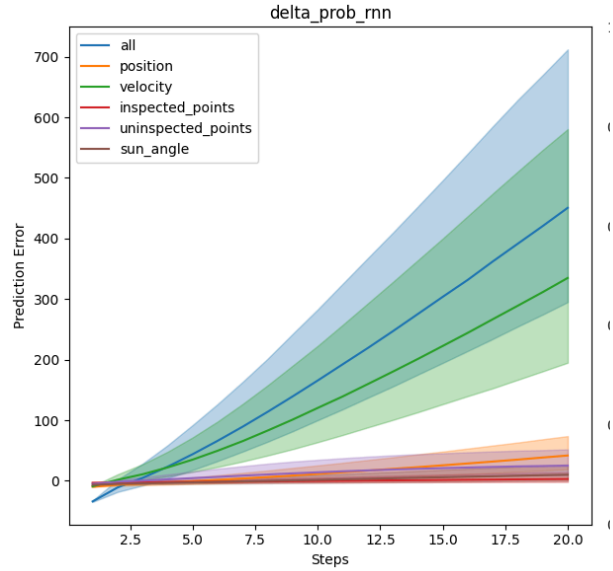


Figure 16.

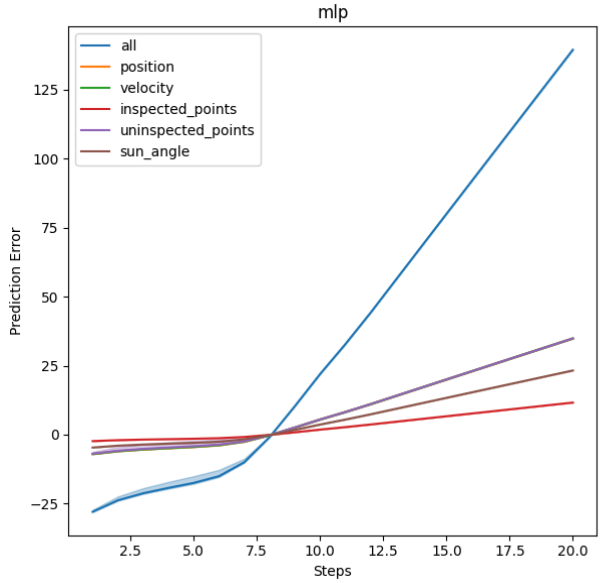


Figure 17.

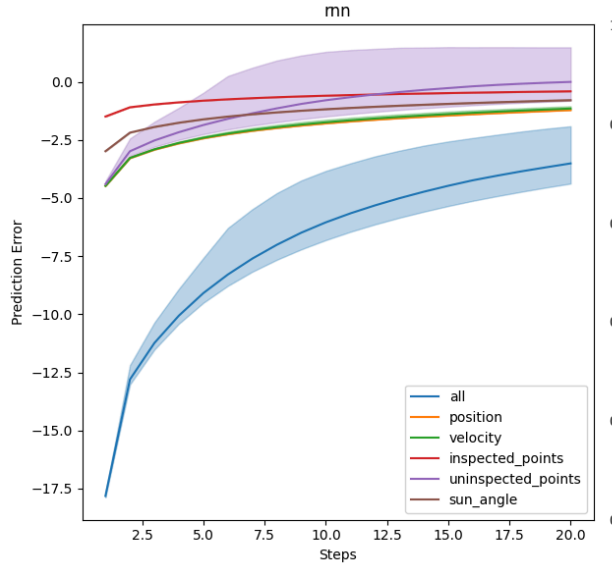


Figure 18.

Conclusions:

Of all the iterations of dynamics models trained throughout the duration of this project, the most successful model was the Delta MLP. Both the Delta MLP and Delta RNN outperformed their absolute counterparts, the MLP and RNN, respectively. These results suggest that compounding error stems, mostly, from the nonlinearity of the data in our training dataset. Further investigation into architecture choices which improve nonlinear capacity could be fruitful. One potential limitation of the models used in this project is the choice of ReLU activation function. Application of richer nonlinearities could increase model capacity and

therefore performance on this dataset. Softmax, Mish, Swish, and symlog activation functions were popular choices in the literature covered in this course (Hansen, 2023) (Ha, 2018) (Hafner, 2023). Simpler options like hyperbolic tangent or sigmoid are also promising candidates.

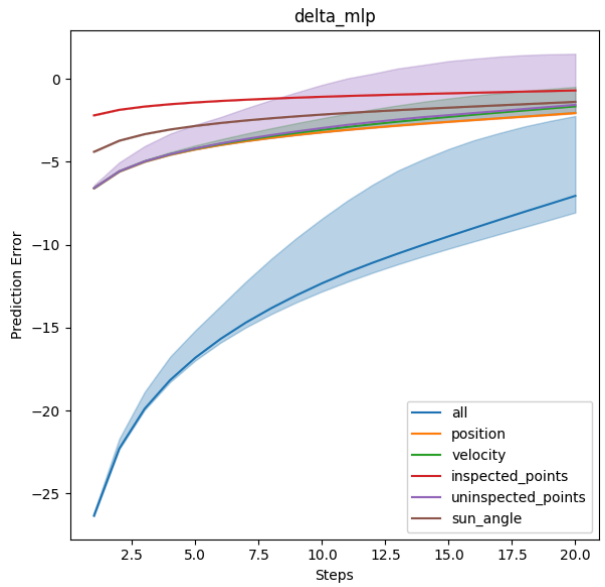


Figure 19.

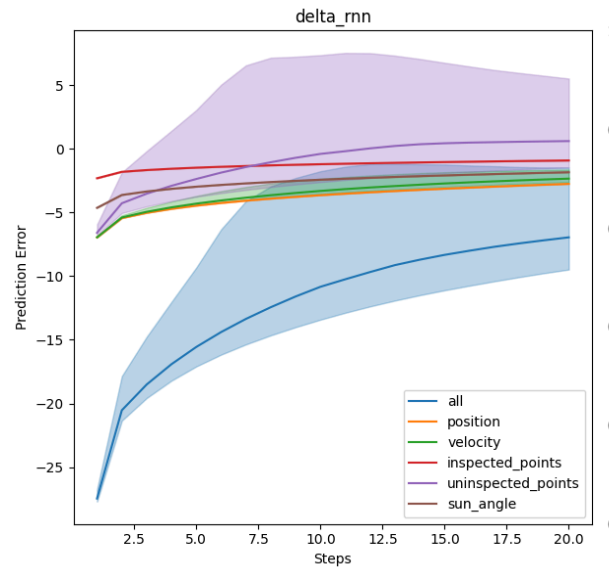


Figure 20.

The training process used in this project failed to conform to some industry best practices. Each model was trained using only one seed, which was kept constant across all models. Given Neural Networks' known sensitivity to random initialization, it is best practice to randomly select a batch of 10 or so seeds and compare the average test performance of each model across all seeds. It is difficult to say if a "bad" seed obfuscated our results for any given model in this project. Also, no exhaustive hyperparameter searches were conducted, potentially omitting undiscovered performance improvements.

The poor multistep prediction results for probabilistic models may be attributable to stochastic sampling of probabilistic model's output distributions. Future work should investigate different approaches, such as using the predicted means as the model's predicted next state. This could be viewed as using the model's "most likely" prediction of the next observations to propagate the prediction sequence. Another approach would be to extend each probabilistic model's input to take in observation values and their log variances. This would allow the model to use more information when computing future probability distributions over the observation space. The training objective would have to be altered to include a multistep loss function, in order to provide a learning signal for log variance inputs (since real data log variances would be zero).

Then again, most of our models would likely benefit from the addition of a multistep loss function. While underrepresented in the literature covered in this course, multistep loss functions have been shown to reduce compounding error at test time. All of the models discussed in this paper were trained solely on single step predictions. We leave the study of multistep loss functions as a promising avenue for future work.

As mentioned previously, the lackluster performance of our constrained models indicates two possibilities: our constraints are overly constrictive and distort the learning signal for the underlying models or some of our constraint assumptions are incorrect. Teasing apart why the constraints degrade performance would require an ablative investigation. Interestingly, the sun angle constraint seems to be the most problematic, based on a comparison of all models' sun angle error (Figure 24). This is unintuitive as the constraint merely normalizes the model's predicted angle to a point on the unit circle (which the actual sun angle is guaranteed to reside on). This is likely due to the constraint's distortion of the learning signal. An interesting investigation would compare prediction errors of a model trained with the constraint and a model trained without the constraint, where the constraint is applied to both models during evaluation.

We can also approach the problem from the opposite direction. Instead of asking what model improvements can accomplish, we can ask "what data alterations can simplify the learning problem"? The representation of features in the observation have a significant impact on the difficulty of a Machine Learning (ML) problem. For example, we found it necessary to normalize all the elements of the observation space to the same scale and convert the sun angle from radians to points on a unit circle. Further amendments might include transforming absolute coordinates into agent-centric coordinates. That is, rather than using the xyz position of the deputy, the position vector can be flipped or rotated so that the relative location of the chief is used. This is a common technique in RL, and it could benefit the learning of an approximate transition function. As discussed above, the uninspected points cluster position consistently contributed disproportionately to the total prediction error of every model. It's possible that this feature could be better represented. Perhaps normalizing the cluster position so that it is always a unit vector could reduce unpredictability. Perhaps exposing more environment information to the dynamics model (like the position and status of every point) would improve performance, although this is a high bandwidth solution. Maybe the inclusion of an "inspected points cluster" would suffice to provide context for the dynamics model to make a better prediction. Again, we leave further feature engineering of the Satellite Inspection environment as a promising direction for future research.

MSE Segment Plots:

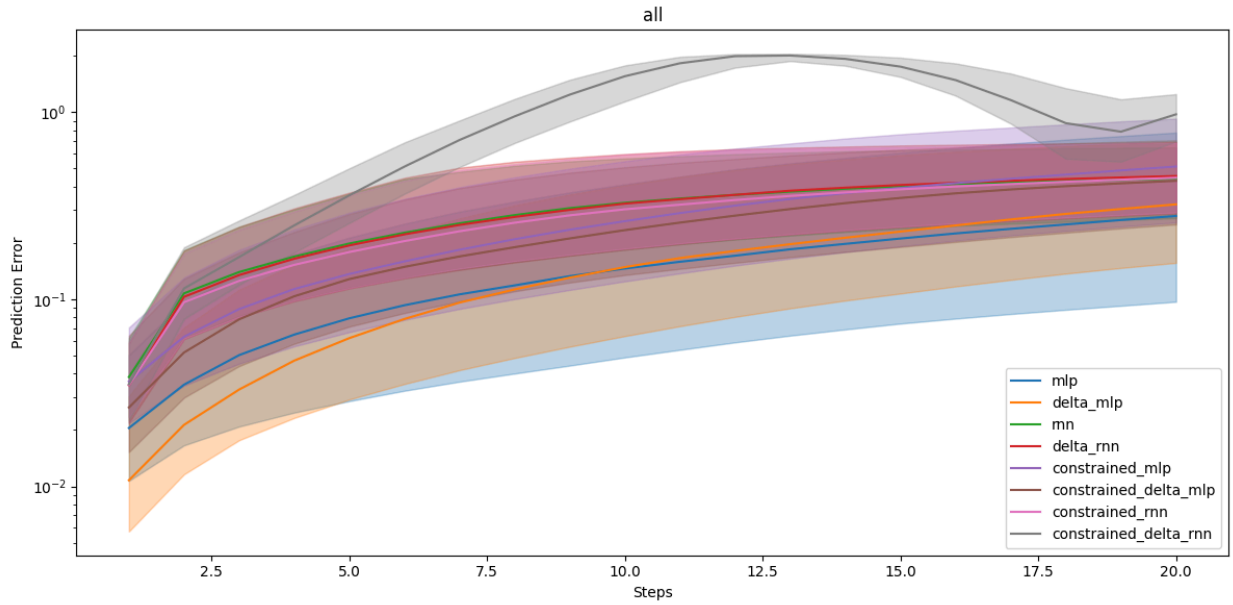


Figure 21.

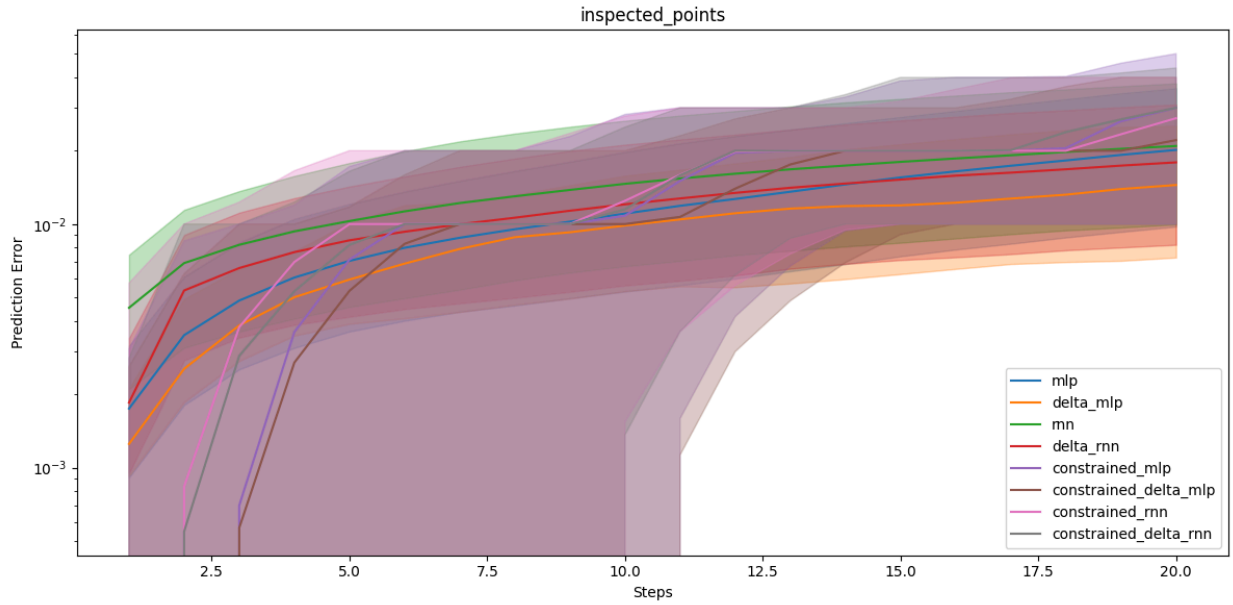


Figure 22.

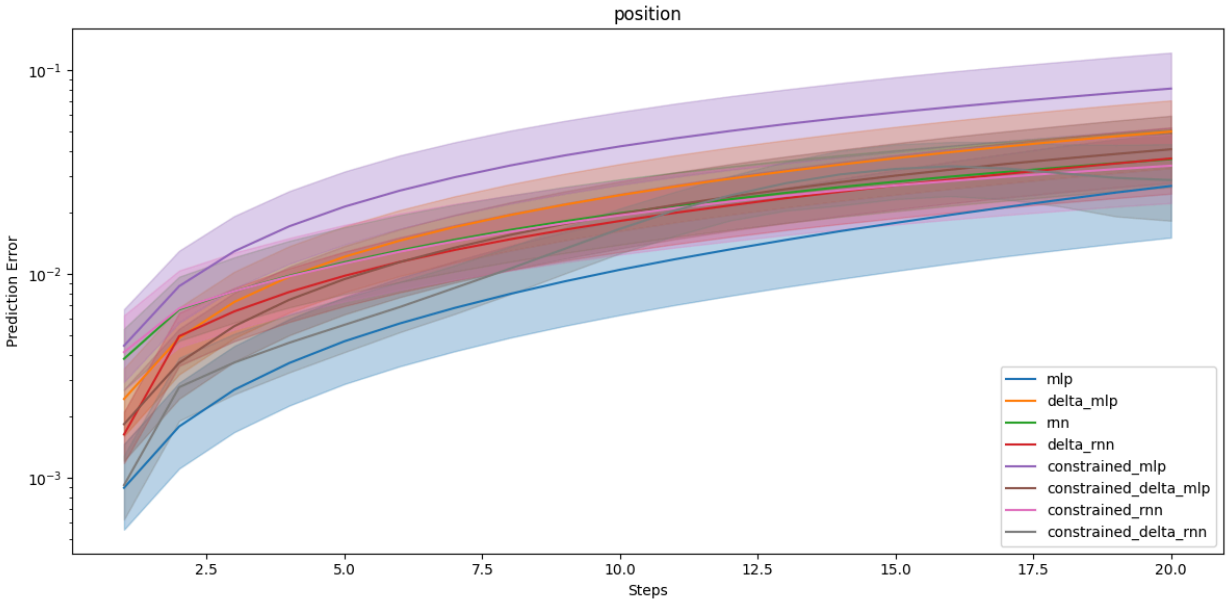


Figure 23.

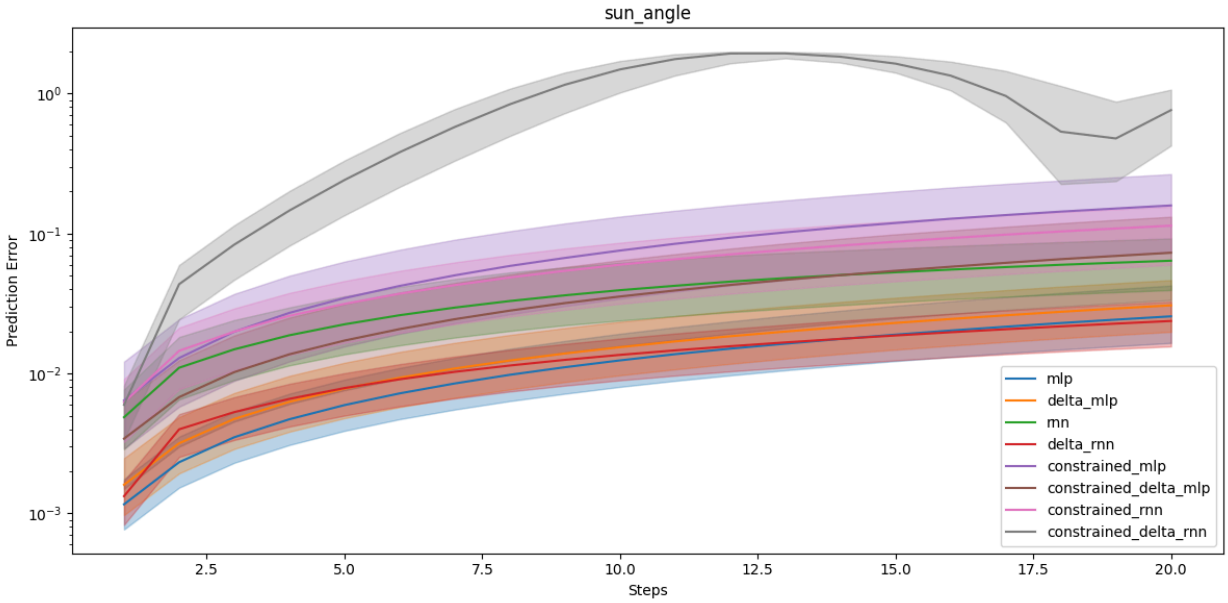


Figure 24.

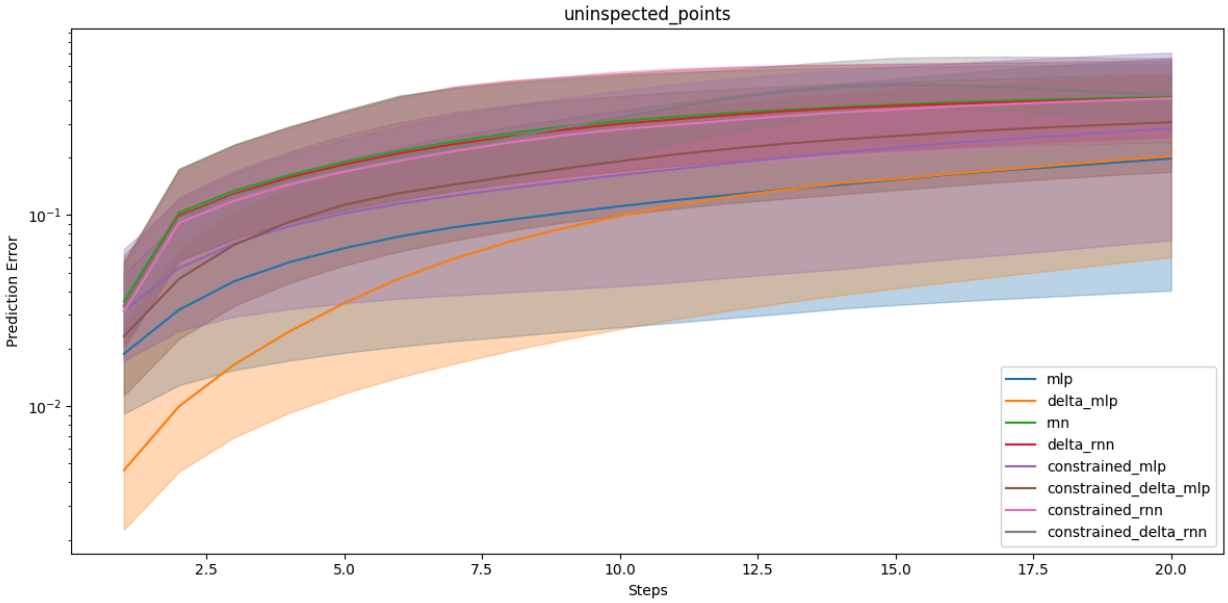


Figure 25.

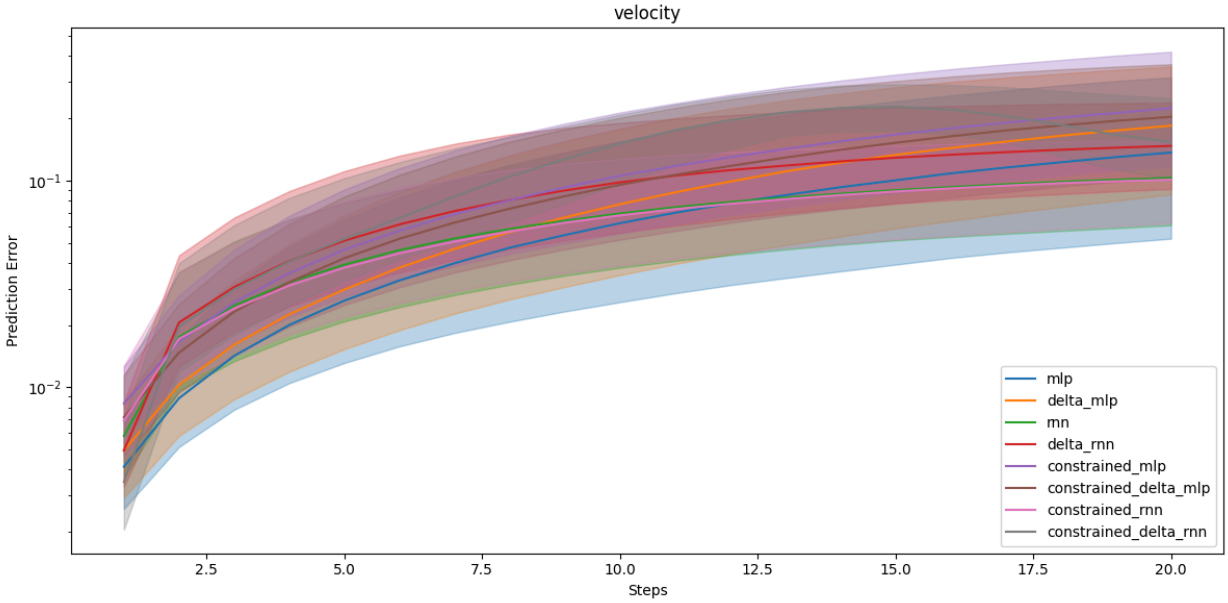


Figure 26.

Table 1.

Paper	World Model	Policy	Planning	Representation	Environments	Time
PlaNet: Learning Latent Dynamics for Planning from Pixels Model-Based Reinforcement Learning for Atari	RSSM	None	CEM	Recurrent state Embedding vector + stochastic one-hot encoding matrix	DeepMind Control Suite	Discrete
Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model	DNN Dynamics function predicts next state and next reward	Probabilistic search policy over discrete action space	MCTS	Embedding vector	Chess, Go, Shogi, and Atari Suite	Discrete
Model-Based Reinforcement Learning for Atari	Deterministic CNN predicts next frame and next rewards + Stochastic VAE estimates posterior distribution of latent state	PPO	None	Embedding vector	Atari Suite	Discrete
Mastering Diverse Domains Through World Models	RSSM	Actor-Critic trained in latent space	None	Embedding vector + stochastic one-hot encoding matrix	Minecraft, Atari suite, Propio control suite, visual control suite, crafter, Bsuite, + DMLab	Discrete
DreamerPro: Reconstruction-Free Model-Based Reinforcement Learning	RSSM (with no reconstruction loss)	Actor-Critic trained in latent space	None	Recurrent state embedding vector clustered into prototypes	DeepMind Control Suite (with noisy backgrounds)	
Imagination-Augmented Agents for Deep Reinforcement Learning	Imagination model predicts next observations and next rewards	CNN takes in rollout encodings and observations and outputs action. Trained with	Simulates rollouts for each action. Rollouts encoded and passed to policy network	observation space	Sokoban + MiniPacman	Discrete

		A3C				
World Models	RNN with Mixture Gaussian Network predicts next latent space	Covariance Matrix Adaptation Evolution Strategy	Background planning	VAE latent embedding vector	Car Racing + DoomViz	Discrete
Woulda, Could, Shoulda: Counterfactually-Guided Policy Search	Structural Causal Model	Return Weighted Regression updates	Background planning	Random Variables and Causal Mechanisms	Partially Observable Sokoban	Discrete
Identifying Reasoning Flaws in Planning-Based RL using Tree Explanations	Deterministic DNN predicts next state and next reward	Dr-DQN	Minimax tree search	observation space	Tug of War Starcraft variant	Discrete
Reinforcement Learning with Prototypical Representations	MLP Encoder + Prototypes set	Max-entropy exploration policy + SAC policy	None	Embedding vector projected onto prototypes	DMC Suite	Discrete
TDMPC-2: Scalable, Robust World Models for Continuous Control	MLP predicts next latent state, separate reward + continue models	Stochastic max-entropy policy	MPPI	Simnormed embedding matrix	DMC Suite, Meta-World, Maniskill2, MyoSuite	Discrete
Learning to Adapt in Dynamic, Real-World Environments Through Meta-Reinforcement Learning	MLP predicts next state	GrBAL meta learning	MPPI	Observation space	Crippled Ant, Disabled half-cheetah, Millirobot	Discrete
Model-Based Policy Optimization with Neural Differential Equations for Robotic Arm Control	MLP predicts state-derivative + differential equation solver	Off-policy SAC	Background planning	State-derivative	Reacher-v2, cheetah-run, DoorOpen	Continuous

Thinker: Learning to Plan and Act	State-Reward RNN + Value-Policy RNN	IMPALA	Tree Search with reset option for search policy	Embedding vector + rollout tree summary	Sokoban	Discrete
When to Trust Your Model: Model Based Policy Optimization	Ensemble of Probabilistic DNNs	SAC	Background planning: Branched rollouts	Observation Space	MuJoCo Control Suite	Discrete
SafeDreamer	RSSM + Cost Models	Actor-Critic trained in latent space	Constrained CEM	Embedding vector + stochastic one-hot encoding matrix	Safety-Gymnasium	Discrete

References

Ravaioli, U. J., Cunningham, J., McCarroll, J., Gangal, V., Dunlap, K., & Hobbs, K. L. (2022, March). Safe reinforcement learning benchmark environments for aerospace control systems. In *2022 IEEE Aerospace Conference (AERO)* (pp. 1-20). IEEE.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT press.

Weidong Huang, Jiaming Ji, Chunhe Xia, Borong Zhang, & Yaodong Yang. (2024). SafeDreamer: Safe Reinforcement Learning with World Models.

Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., ... & Silver, D. (2020). Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, *588*(7839), 604-609.

Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., & Davidson, J. (2019, May). Learning latent dynamics for planning from pixels. In *International conference on machine learning* (pp. 2555-2565). PMLR.

Hansen, N., Su, H., & Wang, X. (2023). Td-mpc2: Scalable, robust world models for continuous control. *arXiv preprint arXiv:2310.16828*.

Buesing, L., Weber, T., Zwols, Y., Racaniere, S., Guez, A., Lespiau, J. B., & Heess, N. (2018). Woulda, coulda, shoulda: Counterfactually-guided policy search. *arXiv preprint arXiv:1811.06272*.

Lam, K. H., Lin, Z., Irvine, J., Dodge, J., Shureih, Z. T., Khanna, R., ... & Fern, A. (2021). Identifying reasoning flaws in planning-based rl using tree explanations. *arXiv preprint arXiv:2109.13978*.

Ha, D., & Schmidhuber, J. (2018). World models. *arXiv preprint arXiv:1803.10122*.

Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2023). Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*.

Racanière, S., Weber, T., Reichert, D., Buesing, L., Guez, A., Jimenez Rezende, D., ... & Wierstra, D. (2017). Imagination-augmented agents for deep reinforcement learning. *Advances in neural information processing systems*, *30*.

Chung, S., Anokhin, I., & Krueger, D. (2024). Thinker: learning to plan and act. *Advances in Neural Information Processing Systems*, *36*.

Deng, F., Jang, I., & Ahn, S. (2022, June). DreamerPro: Reconstruction-free model-based reinforcement learning with prototypical representations. In *International conference on machine learning* (pp. 4956-4975). PMLR.

Walker, J. C., Vértes, E., Li, Y., Dulac-Arnold, G., Anand, A., Weber, T., & Hamrick, J. B. (2023, July). Investigating the role of model-based learning in exploration and transfer. In *International Conference on Machine Learning* (pp. 35368-35383). PMLR.